

PKCERT AI & Software Development Internship

Final Capstone: Phishing Email Inspection Desk

Abdullah Amir

AI and Software Development
28 August 2026

Full code lives in this task's directory (`model_v2/`, `backend/`, `frontend-app/`, `Dockerfile`, `tests/`). A live phishing/safe email classifier: two models trained and evaluated on 18,650 real emails (TF-IDF + Logistic Regression vs. a fine-tuned DistilBERT), a FastAPI backend, a React/Framer Motion frontend built through a structured design workflow, packaged into one Docker image, and deployed live on Render's free tier. The project's actual thesis – deploy the small, cheap model and keep the larger Transformer as a measured comparison, not a live dependency – is a direct, deliberate application of a resource-ceiling finding from an earlier task in this internship, not a fresh discovery.

Part A: Scope, Planning & Integration

A1 – problem and why this project. Phishing email is one of the most common real-world attack vectors, and most people have no fast way to sanity-check a suspicious email beyond intuition. The brief required demonstrable integration of prior-task skills (text processing, embeddings/Transformer modeling, full-stack development) and independent project scoping; a live phishing classifier satisfies all three while giving the project a genuine cybersecurity framing.

A2 – what this replaced. An earlier iteration of this same capstone slot built an identical architecture around movie-review sentiment classification. It was fully working, tested, and deployed before being deliberately replaced with this phishing-detection version – the underlying engineering pattern (two-model comparison, deploy-the-small-one decision, Docker/Render pipeline) carried over almost unchanged; the domain, data, models, and entire frontend were rebuilt.

A3 – tech stack.

Layer	Choice	Why
Data	<code>zefang-liu/phishing-email-dataset</code> (HF datasets)	18,650 real emails, permissively hosted, large enough for a meaningful two-model comparison
Model A	TF-IDF + Logistic Regression (<code>scikit-learn</code>)	Static-embedding-style features, sub-millisecond inference, deployable within a strict free-tier memory budget
Model B	Fine-tuned <code>distilbert-base-uncased</code> (<code>transformers</code>)	Contextual embeddings – the direct comparison point for Model A; evaluated, never deployed
Backend	FastAPI, async, Pydantic validation	Proven pattern reused from an earlier task rather than reinvented
Frontend	React + Vite + Framer Motion	Portfolio-grade presentation for this capstone's evaluative audience; builds to static assets, adding no runtime cost to the deploy target
Deployment	Docker (Node build stage + Python runtime), Render free tier	Single container; the 512MB memory budget is the binding constraint shaping the whole architecture

A4 – hard scope boundary. Detection only. No UI element, endpoint, or copy generates, templates, or suggests phishing content, even illustratively – treated as a non-negotiable constraint, not a style choice, since this is a defensive-security tool.

Part B: Data Preparation, Model Building & Evaluation

B1 – data cleaning, two real issues found. 19 rows had null/empty text. A second, less obvious issue surfaced only after reading actual misclassified examples from a first training run: **533 rows (~2.9%) carried the literal placeholder string “empty” as their entire email text** – a scraping artifact in the source dataset, not real content, and invisible to a naive null check. Both were dropped. Class balance: 11,322 safe vs. 7,328 phishing (~61/39) – handled with `class_weight=balanced` and macro-averaged metrics throughout. Text length is extreme (median 159 words, one 3.5-million-word outlier); raw text is truncated to 20,000 characters before any processing. The dataset ships only a single `train` split, so an 80/10/10 `train/val/test` split was created directly, stratified by label.

B2 – Model A: hyperparameter search (validation set).

<code>max_features</code>	<code>ngram_max</code>	<code>C</code>	Val acc.	Val F1
10,000	1	1.0	0.9751	0.9738
20,000	1	1.0	0.9751	0.9738
20,000	2	1.0	0.9790	0.9779
20,000	2	0.5	0.9751	0.9739
20,000	2	2.0	0.9807	0.9796
30,000	2	1.0	0.9801	0.9790

B3 – Model B: a real resource constraint that changed the training plan. An initial attempt trained on the full ~14,500-row training split at 128 tokens/example, matching the design used elsewhere in this internship. It was killed after **12.8 wall-clock hours still short of finishing epoch 1 of 3** – verified directly via `uptime/free -h` as genuine, sustained system-wide contention (load average consistently above 12 on a 12-core machine, 8GB+ swapped), not a hang or a bug. Rather than keep waiting on an uncontrollable external constraint, the run was restarted at a leaner configuration – a 6,000-example stratified subsample, a 96-token cap, 2 epochs – which completed in under 40 minutes. This is the same category of decision as the project’s Model A/B deployment choice: adapt the plan to a measured real-world constraint.

Epoch	Train loss	Train acc.	Val loss	Val acc.
1/2	0.1695	0.9390	0.0701	0.9768
2/2	0.0394	0.9890	0.0766	0.9785

Validation loss ticks up slightly between epochs while accuracy still edges upward – the same overfitting-begins-early signal seen elsewhere in this internship’s training runs, arriving faster here specifically because the training set is smaller. The saved checkpoint is epoch 2’s (best validation accuracy), not epoch 1’s.

B4 – the comparison, reported honestly.

Metric	Model A (TF-IDF + LogReg)	Model B (DistilBERT)
Test accuracy	98.45%	97.73%
Test macro F1	0.9837	0.9761
Avg. latency	0.20ms/example	46.09ms/example
Artifact size	908KB	267.9MB
Misclassified	28/1,810 (1.5%)	41/1,810 (2.3%)
Deployed live	Yes	No

Model A wins on every axis this time, not just size and speed. Unlike a pattern this same comparison has produced on other tasks (a Transformer trading size/speed for a real accuracy gain), Model A’s 98.45% edges out Model B’s 97.73% here – a fair, disclosed consequence of the resource-constrained retraining in B3, not evidence that Transformers are worse at phishing detection in general. Reviewing Model B’s own misclassified examples shows the same error class as Model A’s: well-crafted promotional/phishing text that reads as formally legitimate, and legitimate emails (a masonic-lodge newsletter, a corporate congratulations note) using unusually formal phrasing. Contextual embeddings did not resolve this ambiguity class under this training budget – a narrower, more honest finding than assuming they always would. This does not change the deployment decision: Model A’s 908KB/0.20ms footprint against Model B’s 267.9MB/46ms was never a close call.

Part C: Backend & Frontend Development, Integration

C1 – API. FastAPI, async routes, Pydantic validation, structured JSON logging, CORS. Three endpoints: `GET /healthz`, `POST /api/v1/predict ({text} → {label, confidence, latency_ms})`, `GET /api/v1/models` (both models’ real measured metrics, read from saved JSON – Model B’s 267.9MB weights are never loaded into this process, only its already-computed results file). Input validation (1–5,000 chars, non-blank) is enforced by Pydantic at the boundary.

C2 – frontend, built through a structured design workflow. React + Vite + Framer Motion, built to static assets and served by FastAPI from the same container – no separate frontend hosting, no Node process at runtime. Designed via the project’s Impeccable workflow: a recorded product context (`PRODUCT.md`), a rolled and weighed visual direction (a *document-authentication-checkpoint* world – an inspection counter under raking light, a stamped verdict – chosen over catalog challengers including a particle- collider event display and a demoscene terminal aesthetic on audience-identification and product-clarity axes), a built implementation, a structured finish review, and a recorded design system (`DESIGN.md`). The predict tool is the first viewport, not a form under marketing copy: pasting text triggers a raking-light scan animation and a rubber-stamped `CLEARED/FLAGGED` verdict, with confidence rendered as real instrument-panel visual weight (bar length, brightness, and glow scaled directly off the measured number, never an inert percentage label).

A genuine design-review finding, fixed. The finish review flagged three uses of a banned UI pattern (a kicker/eyebrow label above a heading) and an identical scroll-reveal animation duplicated across four sections. Both were fixed in a follow-up commit – the kicker framing folded into heading copy, and each section given a materially different motion mechanism (a drawn rule line, a scale-snap, no motion at all, a table row-stagger) so the hero’s scan-to-stamp interaction remains the system’s one authored motion signature. Verified resolved on both desktop and mobile screenshots before shipping.

C3 – scope adherence. Every UI element traces to Part A’s in-scope feature list – no accounts, no history, no multi-class labels, no non-English support, and no interaction that could read as

generating phishing content.

Part D: Testing, Deployment & Documentation

D1 – testing. `tests/test_api.py`, 16 cases via `pytest`: health check, predict happy-path (2), validation errors (5), model-comparison, frontend/static serving (2, including resolving the Vite build’s content-hashed asset filenames from the served HTML rather than hardcoding them), and 4 parametrized clear-case label checks. All 16 pass. **A real bug found before testing even started:** the 533-row “empty” placeholder issue (B1), caught by reading misclassified examples, not a data-quality scan. **A second real bug, found while building:** the built frontend’s Vite output emits root-relative asset paths (`/assets/index-xxxx.js`), which silently 404’d against a fixed `/static` mount prefix – fixed by mounting the built directory at `/` with `html=True` instead.

D2 – deployment, verified directly. Multi-stage Docker build (Node stage builds the frontend, Python stage runs the backend), non-root user, `HEALTHCHECK`, final image **649MB**. Run with a hard `--memory=512m` cap – the exact ceiling that OOM’d an earlier project in this internship – the container used **110MiB (21.48%)** of that budget, with all endpoints and the frontend confirmed working through the capped container itself, not just the uncapped process.

Live cloud deployment succeeded, on Render’s free tier – <https://day29-phishing-inspector.onrender.com> – deployed via Render’s API with GitHub main-branch auto-deploy as the CI/CD pipeline. Verified live, not just built:

```
$ curl https://day29-phishing-inspector.onrender.com/healthz
{"status":"ok","model_loaded":true,"uptime_s":247.55}
```

```
$ curl -X POST .../api/v1/predict -d '{"text": "Verify your account ..."}'
{"label":"phishing","confidence":0.994,"latency_ms":9.08}
```

`/api/v1/models` and the frontend were confirmed live too, both models’ real numbers rendering correctly in the deployed comparison table.

D3 – documentation. This report, `README.md` (setup/usage), Part A’s scoping document, `PRODUCT.md/DESIGN.md` (the design workflow’s product and visual-system records), and the generated architecture diagram together let a third party set up and run this project independently.

Part E: Presentation & Wrap-up

A 9-slide deck (`presentation/slides.html`) covers the problem, the Model A/B decision and why, measured results, the live demo, the deployment-proof certificate, architecture, engineering challenges, and lessons learned – styled consistently with the deployed product’s own design system. The live URL above serves as the working demo. Full internship wrap-up documents (`DAILY_LOGS.md`, `INTERNSHIP_REPORT.md`) and the GitHub repository accompany this submission.